PRÁTICAS EM SISTEMAS OPERACIONAIS

AULA 1

Prof. André Luiz de Souza Paula

CONVERSA INICIAL

O objetivo desta etapa é desenvolver os conceitos básicos de um Sistema Operacional (SO) e seus componentes. Estudar a sua estrutura, as arquiteturas e a relação entre o *hardware* e o *software* nesse contexto.

Temas de estudo:

1. Conceito de Sistema Operacional
2. *Hardware* e *Software*
3. Estrutura de Sistemas Operacionais
4. Arquitetura de Sistemas Operacionais
5. Tipos de Sistemas Operacionais

Antes de entrar na definição de um sistema operacional, é importante refletir sobre como é composto um sistema computacional. Um computador é composto de um ou mais processadores, memória principal, discos, teclado, monitor, interfaces de rede e outros dispositivos. Os programas precisam saber lidar com todos esses elementos e o usuário não precisa conhecer os detalhes de funcionamento da máquina. É nesse cenário que o sistema operacional busca que o usuário tenha uma boa experiência no uso de um computador.

TEMA 1 – CONCEITO DE SISTEMA OPERACIONAL

Todo computador moderno consiste em um grande conjunto de recursos, é um sistema computacional complexo e com componentes que precisam ser controlados de forma eficiente para trazer o resultado esperado pelo programador. O fato é que, mesmo que o computador possa executar um grande conjunto de instruções, ele não é capaz de compreender as necessidades humanas sem que haja uma interface que possa traduzir a linguagem humana em uma linguagem de máquina.

O sistema operacional oculta a complexidade de um sistema computacional e muitos usuários têm dificuldade em identificar o que é um sistema operacional. Portanto, o primeiro exercício que vamos fazer é separar o que é máquina (*hardware* de computador) da parte lógica do computador, ou seja, das instruções (*software*) que ele é capaz de processar.

1.1 Definição de Sistema Operacional

Segundo Tanenbaum, o sistema operacional funciona como uma interface entre o usuário e o computador, contendo duas funções não relacionadas: estender a máquina e Gerenciar Recursos.

Estender a máquina significa que o sistema operacional cria uma abstração das instruções de máquina mais primitivas e permite que o programador possa programar de forma mais simples, protegendo os recursos e oferecendo uma interface de mais fácil programação do que o *hardware*.

O exemplo mais simples de Gerenciamento de Recursos é a forma como o sistema operacional cria uma fila de impressão, para que cada programa tenha seu tempo para poder imprimir. Isso ocorre com o gerenciamento de memória, escalonamento de processos, a forma como os processos utilizam o processador, entre tantos outros recursos que precisam ser gerenciados.

Sistema Operacional (SO) é o *software* responsável pelo controle da execução de todas as tarefas prontas para executar, por gerenciar os recursos computacionais e criar uma interface intermediária entre o usuário e o *hardware* do computador. Os sistemas operacionais modernos acumulam muito mais funções do que é fundamental para o funcionamento do sistema, entre essas funções podemos citar:

- Tratamento de interrupções e exceções
- Criação e eliminação de processos e *threads*
- Sincronização e comunicação entre processos e *threads*
- Escalonamento e controle sobre processos e *threads*
- Gerência de memória
- Gerência de sistema de arquivos
- Gerência de dispositivos de Entrada e Saída (E/S)
- Suporte a redes locais e distribuídas
- Contabilização do uso do sistema
- Auditoria e segurança do sistema

1.2 Formas de distribuição

Os sistemas operacionais podem ser classificados quanto a sua distribuição: *Software* Livre ou Proprietário. A principal diferença é em relação à

sua filosofia, forma de licenciamento, que define as liberdades de uso do *software*, ou as restrições impostas pelo seu desenvolvedor (proprietário).

O *software* proprietário possui um *Copyright*, que é o direito autoral. Por meio desse registro, o autor/proprietário de uma obra pode receber os lucros provenientes de sua venda ou processar aqueles que a utilizam sem autorização ou a copiam. As licenças de *software* proprietário não dão propriedade, elas definem as condições e o direito de uso do *software* por pessoa física ou jurídica.

Outra forma de distribuição é por *Software* Livre, que também está vinculado a um tipo de licença, mas o conceito principal é que essas licenças definem quais as liberdades o usuário terá direito ao utilizar esse *software*. Alguns exemplos de licenças de *software* livre: BSD, MIT, GPL, Apache, LPM e LPE.

Software Livre é uma expressão criada pela *Free Software Foundation,* fundada por Richard Stallmann, através do Projeto GNU. A fundação criou a *General Public License*, que é baseada em quatro liberdades para que o *software* possa ser licenciado como *Software* Livre:

- **Liberdade 0:** a liberdade de executar o programa, para qualquer propósito.
- **Liberdade 1:** a liberdade de estudar como o programa funciona e adaptá-lo para as suas necessidades. Acesso ao código-fonte é um pré-requisito para essa liberdade.
- **Liberdade 2:** a liberdade de redistribuir cópias de modo que você possa ajudar ao seu próximo.
- **Liberdade 3:** a liberdade de aperfeiçoar o programa e redistribuir seus aperfeiçoamentos, de modo que toda a comunidade possa se beneficiar.

TEMA 2 – *HARDWARE* E *SOFTWARE*

Após classificar o sistema operacional como *software*, é necessário compreender o que é *Hardware* e *Software*. Para isso, vamos classificar os diversos tipos de *hardware* de computador e identificar qual é a camada de *software* que atua um sistema operacional. A Figura 1 é apenas uma forma comum que os profissionais da área de tecnologia costumam se referir ao *software* e ao *hardware* e até para entender uma simples piada, ou ironia, é necessário estudar os fundamentos que nos ajudam a mergulhar neste mundo.

Figura 1 – *Hardware* é aquilo que você chuta enquanto *Software* é aquilo que você xinga



Crédito: Drobot Dean/Adobe Stock; Matthew/Adobe Stock.

2.1 *Hardware*

Para simplificar, podemos definir o *Hardware* como sendo a parte física do computador, por exemplo: placas diversas, processador, memória, gabinete, fonte de alimentação, disco rígido, monitor, entre outros.

Cada dispositivo de *hardware* do computador pode ser separado por função e classificado como:

1. **Dispositivos de entrada:** teclado, *mouse*, *scanner*, *webcam*, entre outros dispositivos que permitem a entrada de dados para que sejam processados.
2. **Dispositivos de saída:** monitor, projetor, impressora e demais dispositivos que possam interagir com o usuário, apresentando o resultado do processamento.
3. **Dispositivos de entrada e saída (híbridos):** tela *touchscreen*, placa de rede (*download/upload*), placas de som, ou seja, são dispositivos com capacidade de alimentar um sistema de processamento e, ao mesmo tempo, devolver o resultado do processamento, interagindo com o usuário para as duas funções anteriores.
4. **Dispositivos de armazenamento:** são dispositivos com capacidade de memória, neles se pode armazenar e recuperar um dado/informação. Por exemplo: disco rígido, CD, DVD, *pendrive*, cartão de memória, memória RAM, memória cache e registradores.

5. **Dispositivos de processamento**: dispositivos com um conjunto de instruções relacionadas a uma arquitetura, com capacidade de processamento, podendo ser um simples controlador, microcontrolador ou processador.

A Figura 2 representa um sistema de processamento de dados, os dispositivos que foram discutidos anteriormente estão classificados de acordo com esse modelo:

Figura 2 – Sistema de Processamento de Dados



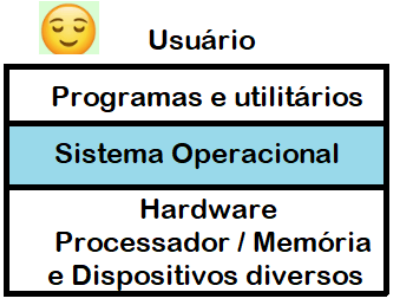
Fonte: Paula, 2023.

2.2 *Software*

O *software* de um computador pode ser definido como um conjunto finito de instruções organizadas de forma lógica, não ambíguo, com objetivo de ser processado pelo *hardware* do computador, ou simplesmente definido como sendo a representação de um algoritmo. O *software* está presente desde a fabricação do *hardware*, nesta fase, definido como *firmware*, um tipo de *software* dedicado e com instruções específicas de entrada e saída para esse equipamento. E chega até programas mais complexos como suítes de desenvolvimento, programas de escritório, jogos e utilitários.

Na Figura 3, o Sistema Operacional é classificado como uma camada de *software* intermediária entre o *hardware* e os programas utilizados pelo usuário.

Figura 3 – Camadas de *Software*



Fonte: Paula, 2023.

É importante observar que o sistema operacional é uma camada intermediária, portanto, precisa oferecer serviços ao usuário e utilizar-se de recursos oferecidos pelo *hardware*. Nesse sentido, o sistema operacional é obrigado a oferecer uma linguagem de alto nível ao usuário, precisando conhecer detalhes do *hardware* para que possa acessar seus recursos com linguagem de máquina (ou linguagem de baixo nível).

- **Linguagem de alto nível:** a principal vantagem dessa linguagem é a facilidade de aprendizagem e a produtividade para o desenvolvimento de novos programas. A ideia é se aproximar da linguagem natural (compreensão humana). É uma linguagem humanizada, mais intuitiva, amigável e de fácil aprendizado.

A principal característica é a abstração dos conceitos e simplificação dos comandos, por exemplo, *delete*, *print*, nas quais são instruções comuns dessas linguagens e executam o que o termo sugere, que é deletar algum dado ou mostrar uma informação na tela.

Exemplos de linguagens de alto nível: Node.js, Javascript, Java, Ruby, C#, Swift, Python e PHP.

- **Linguagem de baixo nível:** a vantagem dessa linguagem é o tempo de processamento. A arquitetura do dispositivo é mais aproveitada com esse tipo de linguagem e oferece melhor desempenho. Também conhecida como linguagem de máquina, possui instruções mais complexas e comandos nada intuitivos porque o objetivo é que seja interpretada diretamente pelo *hardware*.

A linguagem de baixo nível possui instruções diretas para o processador e a principal linguagem de baixo nível é o *Assembly*.

TEMA 3 – ESTRUTURA DE SISTEMAS OPERACIONAIS

A compreensão da estrutura de um sistema operacional passa pela abstração de um sistema computacional visto como uma máquina de multiníveis. Cada camada de abstração de *hardware* ou *software* torna evidente a função do sistema operacional. Os modos de acesso e outros conceitos relacionados à estrutura dos sistemas operacionais serão estudados.

O núcleo (*kernel*) do sistema operacional é constituído de um conjunto de rotinas, oferecidas como serviços aos usuários, às suas aplicações e utilitários do sistema para gerenciamento dos recursos. Ele não executa as tarefas de forma sequencial, mas de forma concorrente de acordo com as prioridades ajustadas, algumas instruções de *hardware* e de sistema podem ser privilegiadas em relação aos programas e aplicativos que rodam em modo usuário.

3.1 Modos de acesso

O sistema operacional deve proteger os recursos que administra, em uma gravação de um arquivo em disco ou uma leitura da memória, por exemplo, as aplicações não devem ter acesso direto às instruções que fazem entrada e saída (E/S) com esse tipo de dispositivo, para resolver esse problema opera com duas formas:

- **Modo núcleo**: o kernel do sistema operacional opera qualquer tipo de instrução que a máquina (Arquitetura/ISA) seja capaz de executar.
- **Modo usuário**: aos processos de usuário, apenas um subconjunto de instruções da máquina fica disponível. São bloqueadas, em particular, aquelas que afetam o controle da máquina e realizam entrada e saída (E/S).

3.2 *System Calls*

São como uma porta de entrada para o acesso ao núcleo do sistema operacional e a seus serviços. Esse termo é usado em sistemas Unix, no ambiente Microsoft chamamos de APIs. O conjunto de serviços oferecidos por um sistema operacional cria uma biblioteca de chamadas de sistema com nomes, parâmetros e formas de ativação específicos. Nos sistemas POSIX e similares, as chamadas de sistema mais usadas são *close, execve, fork, wait, kill, open, read, write, clone, nice, exit, send, receive, signal* e *ioctl*. Os sistemas operacionais atuais têm centenas de chamadas de sistema. Por exemplo, os sistemas GNU\Linux têm quase 300 chamadas de sistema diferentes.

Cada arquitetura tem sua própria tabela de *System Calls*, estas são algumas chamadas em linguagem de alto nível:

pid = fork() – Essa chamada cria um processo (filho) idêntico ao processo pai.

pid = waitpid(pid, &statloc, options) – O processo pai volta a rodar somente após o término do processo filho, ou seja, o processo pai aguarda o filho ser finalizado.

s = execv(name, argv, enrionp) – É realizado, no processo, a substituição da imagem do núcleo.

exit(num) – Termina um processo e retorna um *status* (um número identificado algo, como sucesso, erro etc.).

***fd = open (file*, abertura ...)** – Abre um arquivo (leitura, escrita ou os dois);

s = close(fd) – Fecha o arquivo anteriormente aberto.

n = read (fd, buffer, nbytes) – Lê dados do arquivo 'fd' no '*buffer*', de '*nbytes*' valores de *bytes*.

***n = write(fd, buffer, nbytes*)** – Analogamente ao *read*, mas escreve.

position = lseek(fd, offset, whence) – Move o ponteiro de 'fd'.

s = stat(name, &buf) – Retornar informações do arquivo.

s = chdir(nome) – Altera o diretório em que se está trabalhando.

s = kill(pid, sinal) – Envia um sinal 'sinal' para o processo de PID '*pid'.*

seconds = time(&seconds) – Essa chamada retorna o tempo, em segundos, desde 1o de janeiro de 1970.

s = chmod(nome, modo) – Altera o sistema de proteção do arquivo 'nome'.

s = mkdir(nome, modo) – Cria uma pasta (diretório).

s = rmdir(nome) – Apaga o diretório de nome "nome", previamente criado.

s = mount(special, nome, flag) – Cria um sistema de arquivos (como um dvd ou *pendrive*).

s = umount(special) – Desmonta o sistema de arquivos previamente criado.

s = link(nome1, nome2) – Cria a entrada 'nome2' apontando pra 'nome1'.

s = unlink(nome) – Desfaz a entrada 'nome'.

3.3 Sistemas monolíticos

Os primeiros sistemas UNIX que foram construídos utilizaram a simplicidade dessa arquitetura, em que diversas rotinas são escritas e ligadas a um único programa binário executável. A manutenção era dificultada porque cada rotina tem sua interface (parâmetros e resultados), podendo interagir com qualquer outra rotina.

Arquitetura Monolítica é um sistema único, não pode ser dividido, que é executado em um único processo.

3.4 Sistemas de camadas

O primeiro sistema de camadas foi construído por Dijsktra (1968) na Holanda, são seis camadas funcionais:

5	O operador
4	Programas de usuário
3	Gerenciamento de entrada/saída
2	Comunicação operador-processo
1	Memória e gerenciamento de tambor
0	Alocação do processador e multiprogramação

A ideia principal é utilizar técnicas de programação estruturada modular, onde o sistema é dividido em níveis sobrepostos. Nesses sistemas, as funções oferecidas por uma camada somente poderiam ser acessadas pela camada superior:

- **Camada 0 (zero):** responsável pelo gerenciamento do processador, alternando a alocação entre processos através de interrupções ou quando expiravam os temporizadores. Acima da camada 0 (zero), o sistema implementa processos sequenciais, cada um dos quais podia ser programado sem ser necessário preocupar-se com o fato de que múltiplos processos estavam executando em um único processador. A camada 0 (zero) era responsável por implementar uma multiprogramação básica no processador.
- **Camada 1:** camada de gerenciamento da memória. Responsável pela alocação de espaço para os processos da memória principal e em um tambor (meio magnético de armazenamento de dados) com 512K de palavras utilizado para armazenar partes do processo (páginas) para os quais não havia lugar na memória principal. Acima da camada 1, os processos não tinham que se preocupar com o fato de eles estarem em memória ou no tambor, o *software* da camada 1 cuidava de assegurar que as páginas fossem levadas para a memória sempre que fossem necessárias.
- **Camada 2:** camada intermediária de comunicação entre o console do operador e cada processo.

- **Camada 3:** camada de gerenciamento de dispositivos de entrada e saída.
- **Camada 4:** camada dos programas e utilitários de usuários que fazem interface com o usuário.
- **Camada 5:** usuário do sistema operacional.

3.5 Sistemas de microkernel

O conceito da arquitetura de microkernel tem o objetivo de tornar o núcleo menor, sendo o mais simples possível. Os serviços do sistema são executados através de processos, fora do núcleo, e cada um é responsável por oferecer um conjunto de funções ao usuário. Neste modelo, somente o microkernel é executado em modo kernel e todos os servidores podem ser executados em modo usuário.

Sistemas GNU/Linux utilizam um meio termo entre sistemas monolíticos e microkernel, usando o conceito de módulos que se ligam dinamicamente ao Kernel em tempo de execução e são executados com o Kernel dando funcionalidades adicionais ao sistema. Esse método modular permite um Kernel mais enxuto, política usada no microkernel, mas em uma execução única e em modo Kernel como propõe dos sistemas monolíticos.

Não há uma definição clara de qual é o melhor modelo de implementação, aplicações específicas podem se beneficiar mais de um tipo do que do outro. Um exemplo que diferencia um modelo do outro é o sistema de arquivos. Em um microkernel, o sistema de arquivos é um componente isolado, já no monolítico, faz parte do kernel, assim como o *drivers*, dependendo da filosofia de cada sistema.

TEMA 4 – ARQUITETURA DE SISTEMAS OPERACIONAIS

A arquitetura de sistemas operacionais está relacionada com a implantação do ambiente, compartilhamento de recursos e função do sistema operacional. Novos conceitos são apresentados, como exemplo: Módulo Cliente do Sistema Operacional de Rede (SORC) e Módulo Servidor do Sistema Operacional de Rede (SORS).

Algumas arquiteturas serão abordadas nesta etapa com o objetivo de simplificar o estudo, sem a intenção de esgotar o assunto ou explorar cada arquitetura de forma mais ampla.

4.1 Arquitetura *Peer-to-Peer*

Peer-to-Peer (P2P) é uma arquitetura colaborativa, sem necessidade de se manter um servidor dedicado, isso porque não há um servidor central, sendo que os clientes desse sistema podem atuar como uma espécie de servidores. Os arquivos que percorrem toda a rede *Peer-to-Peer* estão divididos em uma quantidade igual de blocos em cada nó dos computadores dos usuários. Neste modelo, os dispositivos (computadores) que fazem parte da arquitetura devem manter um Módulo Cliente do Sistema Operacional de Rede (SORC) e um Módulo Servidor do Sistema Operacional de Rede (SORS).

As técnicas utilizadas pelas redes *Peer-to-Peer* são implementadas a nível de aplicação, ou seja, independentemente da arquitetura da rede física os nós são conectados por ligações virtuais/lógicos. Essas ligações lógicas são usadas para encontrar e reconhecer os pares com objetivo de tornar a rede *Peer-to-Peer*.

Além do desafio de interligar os nós da rede *Peer-to-Peer* existe o desafio do modelo e dos algoritmos a serem utilizados. Entre os modelos utilizados, estão:

- **Diretório centralizado:** é criado um diretório central onde os nós registram o conteúdo que oferecem para compartilhar e quando existe uma solicitação o nó central responde associando o pedido ao melhor nó no diretório que o corresponde.
- **Busca por inundação:** é um modelo considerado P2P puro, não existe um sistema central de gerenciamento, para atender as requisições são realizadas buscas enviando mensagem para todos os nós (por inundação). Apesar de não parecer eficaz, funciona bem em comunidades menores e não contém um único ponto de falha.
- **Roteamento de documento:** inspirado no roteamento tradicional da rede. utiliza identificadores (ids) para cada nó, a grande dificuldade desse modelo é que para ser “roteado” ao conteúdo compartilhado é necessário saber previamente o ID do documento para poder fazer a requisição.

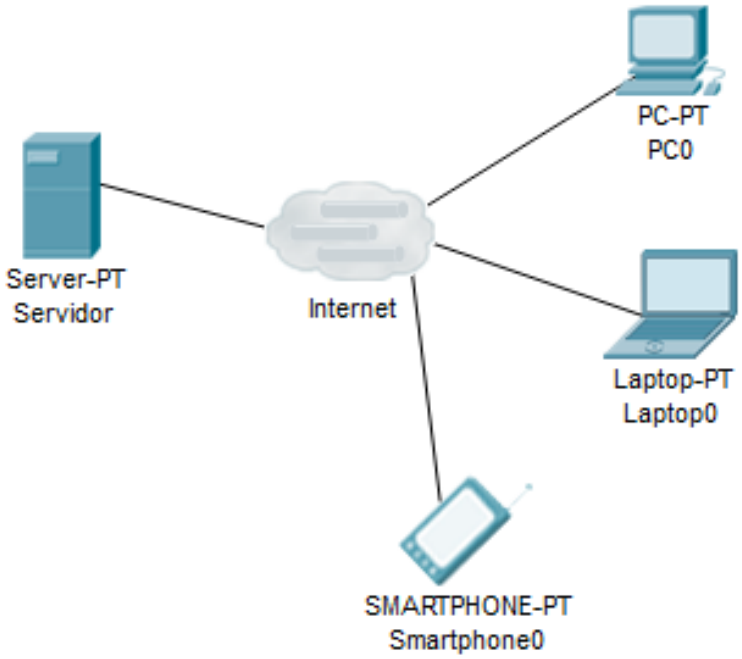
4.2 Arquitetura Cliente-Servidor

No formato de sistema cliente-servidor, um único servidor pode garantir o funcionamento da transferência de informações. O conceito de cliente-servidor

refere-se, portanto, a um modelo de comunicação que vincula vários dispositivos através de uma rede. O cliente solicita ao servidor um recurso, que fornece uma resposta. Neste modelo, os dispositivos (computadores) que fazem parte da arquitetura devem manter um Módulo Cliente do Sistema Operacional de Rede (SORC) ou um Módulo Servidor do Sistema Operacional de Rede (SORS).

É, portanto, uma arquitetura centralizada e pode ser representada conforme a Figura 4, onde somente o servidor compartilha recursos ou serviços a rede, aqueles que consomem o recurso compartilhado são chamados de clientes:

Figura 4 – Arquitetura Cliente x Servidor



Fonte: Paula, 2023.

4.3 Servidor Dedicado x Servidor não Dedicado

Um servidor dedicado, em que o sistema operacional é instalado diretamente no *hardware*, é o próprio servidor da sua organização e contém apenas seus dados. Um servidor não dedicado significa que seu servidor está "hospedado" em um ambiente compartilhado com outras organizações separadas, e o sistema operacional é instalado sobre um servidor virtualizado que possui uma camada intermediária chamada de Hipervisor (camada de virtualização, ou monitor de máquina virtual).

Existem várias técnicas para compartilhar recursos, entretanto, a virtualização de servidores é a técnica que permite que cada sistema operacional (e versão) não precise de um *hardware* dedicado (servidor dedicado). Existem vários ganhos com relação à dedicação exclusiva de um servidor para um único sistema operacional porque isso representa menor custo de instalação, manutenção, refrigeração, energia e suporte técnico para sua administração.

4.5 Sistemas distribuídos

Podemos definir que sistemas distribuídos são aqueles que compartilham recursos: arquivos, dispositivos de E/S, memória e processador.

O grande desafio deste modelo é o compartilhamento de memória e processador. Vários processadores disputando pelo acesso a uma área de memória e umas das técnicas mais utilizadas é o Acesso Não Uniforma a memória (NUMA, do inglês *Non Uniform Memory Access*) em que as áreas de memória são separadas e classificadas e algumas áreas podem ser acessadas mais rapidamente que as outras, mas, durante o uso, a área de memória estará acessada a um processador (CPU).

Com relação ao compartilhamento de processadores (CPU), o desafio é fazer com que os programas possam dividir problemas que podem ser resolvidos de forma paralela, sem nenhuma dependência e podemos ter uma solução mais rápida do que se esses problemas fossem resolvidos por uma única CPU.

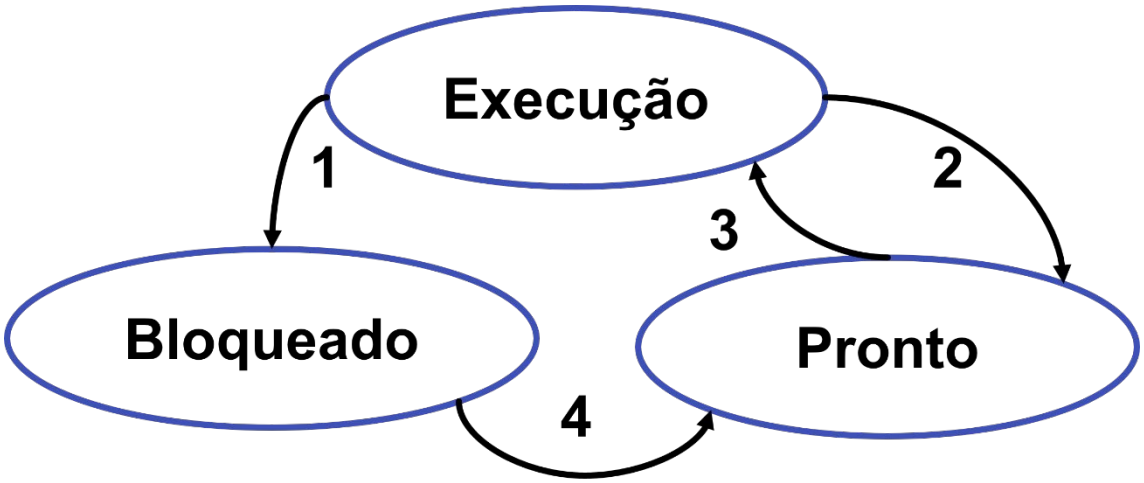
TEMA 5 – TIPOS DE SISTEMAS OPERACIONAIS

Os diversos sistemas operacionais podem ser classificados por várias características: número de tarefas, número de usuários, sistema usuário, sistema servidor, interfaces para o usuário, distribuído, de rede, entre outas características. Os sistemas serão separados por tipos de acordo com cada classificação.

Uma das principais abstrações do sistema operacional é transformar um programa armazenado em uma tarefa a ser executada. Para isso, as tarefas são processos, ou seja, programas em execução. Os processos são preparados executados na ordem e prioridade definida pelo sistema operacional, esta função do sistema operacional se chama Escalonamento de Tarefas

A Figura 5 representa os estados que um processo pode assumir durante a execução de uma tarefa.

Figura 5 – Estados de processos



Fonte: Paula, 2023.

5.1 Sistemas monotarefa

São sistemas que permitem que apenas uma tarefa seja executada por vez, sendo que a próxima tarefa só é chamada no final da execução da tarefa atual. Esse sistema tem um algoritmo de alocação de tarefas extremamente simples, controlando os estados de execução de uma única tarefa, neste modelo, a memória era dividida apenas em partição de tarefa e partição de sistema. Nesse sistema, enquanto o processador aguardava por um evento de E/S, permanecia ocioso, não havendo compartilhamento de recursos.

5.2 Sistemas multitarefa

A partir do conceito de *TimeSharing* (tempo compartilhado), e o particionamento de memória, abriu-se as portas para a multiprogramação, que permitiu o desenvolvimento os sistemas de tempo compartilhado, o pseudo-paralelismo e as diversas formas de paralelismo que encontramos hoje nos sistemas operacionais atuais. Houve uma considerável redução de tempo na execução de tarefas, pois o processador pode se ocupar com outro processo caso haja um evento de E/S.

Um sistema multitarefa pode suportar uma ou mais formas de gerenciamento de tarefas:

- **Sistemas multiprogramados Bach:** utilizados na década de 1960, foi o primeiro sistema multiprogramado desenvolvido. Neste tipo de sistema

multitarefa as tarefas (programas), eram armazenadas em cartões perfurados. O processamento era feito através de lotes de tarefas enfileiradas e processadas de forma sequencial, ou seja, uma tarefa só era processada quando a tarefa à sua frente terminava todo seu processamento. Cada tarefa era responsável por uma única função, como um cálculo qualquer. Como as tarefas eram colocadas enfileirados, após o fim de uma tarefa, o segundo programa era automaticamente iniciado, sem a necessidade da troca manual. Sem nenhuma interatividade com o usuário.

- **Sistemas multiprogramados de tempo compartilhado:** esses sistemas também são conhecidos por sistemas *TimeSharing*. As execuções das tarefas são divididas em fatias de tempo (*time slice*), e o processador é cedido a cada tarefa nesses pequenos intervalos de tempo até que seja finalizada. Os programas em execução compartilham o uso do processador e se alternam, cada programa em execução fazendo uso do seu intervalo de tempo. Desta forma, o tempo de uso do processador é compartilhado pelas várias tarefas em execução pelo sistema operacional. Quando um programa termina seu intervalo de tempo de uso do processador, ele é interrompido pelo sistema operacional, sendo substituído por outro programa e fica aguardando um novo intervalo de tempo para ter sua execução reiniciada do ponto onde foi interrompido.
- **Sistemas multiprogramados de tempo real:** para cada tarefa a ser executada, uma prioridade é atribuída, e a escolha da tarefa a ser executada segue as prioridades que foram previamente estabelecidas. Os sistemas de tempo real podem impactar em tarefas com menor prioridade, pois sua execução pode ser atrasada em função de tarefas com maior prioridade. As tarefas não são executadas de forma instantânea como se imagina, ainda depende dos tempos de processador para sua execução. Uma tarefa permanece em execução pelo processador o tempo que for necessário, até que outra tarefa com maior prioridade seja iniciada. Esses sistemas são usados para aplicações críticas como controle de tráfego aéreo, sistemas de defesa, usinas nucleares, entre outras aplicações. Nestes sistemas, o tempo de resposta é crítico, aumentando o controle de operações.

5.3 Sistemas monousuário

Os sistemas de um único usuário não permitem o acesso simultâneo e processos de diferentes usuários, exemplos: MS-Windows 98, iOS, Android, Arduino, entre outros.

5.4 Sistemas multiusuário

São sistemas que permitem ambientes interativos que possibilitam que diversos usuários se conectem simultaneamente, exemplos: Sistemas UNIX, MS-Windows XP (assim como todas as versões posteriores) e sistemas GNU/Linux.

5.5 Sistemas monoprocessados

São sistemas operacionais que controlam uma única Unidade Central de Processamento (UCP ou CPU – sigla em inglês). Esses sistemas não têm a capacidade de distribuir tarefas em mais de um processador.

5.6 Sistemas multiprocessados

Caracterizam-se por possuir duas ou mais processadores (CPUs) e permitir que vários programas sejam executados ao mesmo tempo, são conhecidos como:

- **Fortemente acoplado:** os diversos processadores (CPUs) que integram esse sistema compartilham de uma única memória física e dispositivos de E/S gerenciados por um único sistema operacional. São exemplos os sistemas *SMP* (*Symmetric Multiprocessors*) e *NUMA* (*Non-Uniform Memory Access*). Por exemplo: um computador, ou servidor, com mais de um processador físico e vários núcleos de processamento pode ser considerado um sistema fortemente acoplado.

- **Fracamente acoplado:** caracteriza-se por integrar dois ou mais sistemas operacionais, também conhecido como multicomputadores. Os diversos tipos de *clusters* e *grids* podem ser utilizados para realizar processamento distribuído.

Um *cluster* pode ser definido como um conjunto de servidores interconectados, com objetivo de atuar como se fossem um único sistema e trabalham em conjunto para realizar tarefas de forma mais eficiente e escalável. Esses sistemas podem oferecer características como alta disponibilidade, balanceamento de carga e processamento paralelo.

5.8 Sistemas servidores

Definido por Tanenbaum como sendo sistemas que servem múltiplos usuários e em rede, permitindo o compartilhamento de *hardware* e *software*. Normalmente, trabalhando na arquitetura Cliente-Servidor, podem oferecer serviços de impressão, autenticação, resolução de nomes, arquivos, compartilhamento de Internet, armazenamento e execução de aplicações entre várias outras tarefas.

Os servidores podem ser dedicados ou compartilhados. O termo *servidor* também define um recurso dentro de um sistema computacional capaz de processar aplicações, prestar serviços e armazenar dados. Esses sistemas podem ser físicos ou virtuais, estarem instalados local ou remotamente. Normalmente, são escaláveis e possuem alto poder de processamento.

Exemplos de sistemas operacionais servidores: UNIX (AIX, BSD, Solaris, FreeBSD, entre outros), MS-Windows Server [NT | 2000 | 2002 | 2003 | 2012 | 2013 | 2016 | 2019 | 2022] e GNU/Linux (Slackware, RedHat, Debian, Suse, Oracle etc.).

5.9 Sistemas de computadores pessoais

Menos robusto que os sistemas servidores, seu principal objetivo é oferecer uma melhor experiência de uso dos computadores por seus usuários. Neste sistema, a Interface com o usuário é essencial, além de permitir o correto funcionamento de uma grande quantidade de programas e utilitários.

Os sistemas operacionais modernos investem muito na humanização da tecnologia através da Interação Humano-Computador (IHC). A IHC estuda a melhor forma de facilitar o diálogo entre usuários e máquinas, transformando comandos complexos em experiências intuitivas e satisfatórias. Neste contexto, é fácil perceber o quanto as telas têm facilitado a vida do usuário, os sistemas operacionais são os principais responsáveis por desenvolver ferramentas de acessibilidade, de interatividade. Seu objetivo é:

- Atender desejos e necessidades dos usuários.
- Se preocupar com as habilidades ou possíveis limitações físicas do usuário.
- Compreender como funciona o sistema de percepção do usuário.
- O que os usuários acham atrativo ou agradável ao interagir com computadores.

Exemplos de sistemas para usuários (Desktops): MS-Windows [7 | 10 | 11], iOs, Android e GNU/Linux (Slackware, Fedora, Debian, OpenSuse,).

FINALIZANDO

Com o objetivo de compreender os conceitos, arquiteturas e tipos de sistemas operacionais, foi realizada uma revisão dos conceitos que permitem desenvolver uma visão sistêmica sobre o funcionamento dos Sistemas Operacionais.

Vamos listar, a seguir, os principais conceitos revisados nesta etapa:

1. **Sistema operacional:** *software* responsável por gerenciar os recursos de um computador e oferecer uma interface amigável (abstração) para seu usuário, através de um conjunto de programas.
2. ***Software*:** conjunto de instruções organizadas de forma lógica, não ambígua, para ser executado pelo computador em forma de tarefas (processos), que visam atingir um objetivo bem definido.
3. ***Software* livre:** licença em que o autor define liberdades de uso do *software*, a principal licença de *software* livre é a *General Public License* – GPL da *Free Software Foundation*.
4. ***Software* Proprietário**: licença que restringe o uso do *software* através de um registro de *Copyright*.
5. **Linguagem de alto nível:** linguagem voltada para o programador, com instruções mais próximas da linguagem natural.
6. **Linguagem de baixo nível:** linguagem de máquina, contém instruções suportadas pela arquitetura do processador.
7. ***Hardware*:** dispositivos físicos que compõem um computador, como placas, teclado, *mouse*, monitor, entre outros.
8. **Dispositivo de entrada:** tipo de dispositivo que possui interação com o usuário, ou outro computador, para entrada de dados.

9. **Dispositivo de saída:** tipo de dispositivo que possui interação com o usuário, ou outro computador, para saída do resultado de um processamento.
10. **Dispositivo de processamento:** tipo de dispositivo capaz de executar um conjunto de instruções para o tratamento dos dados.
11. **Dispositivo de armazenamento:** tipo de dispositivo capaz de armazenar e recuperar informações.
12. **Dispositivo de entrada e saída:** tipo de dispositivo que possui interação com o usuário, ou outro computador, para entrada e saída de dados.
13. **Modo núcleo:** modo privilegiado através do qual o sistema operacional tem acesso completo ao computador, todas as partes do *hardware*.
14. **Modo usuário**: modo restrito que limita as ações do usuário para garantir a segurança e manter a integridade do sistema.
15. ***SystemCalls*:** uma chamada de sistema permite que um aplicativo de usuário solicite ações (conjunto de bibliotecas) que requerem privilégios especiais para serem executadas.
16. ***Peer-to-Peer*:** arquitetura descentralizada na qual os computadores conectados ao sistema funcionam também como servidores.
17. **Cliente x servidor:** arquitetura centralizada onde normalmente o servidor é dedicado, e é projetado especificamente para gerenciar e fornecer serviços aos clientes.
18. **Sistemas distribuídos:** são sistemas que compartilham recursos computacionais como processamento, memória, dispositivos de E/S e arquivos.
19. ***TimeSharing*:** é uma técnica que permite compartilhamento de processamento entre diversas tarefas, o compartilhamento é **baseado** em fatias de tempo (*time slice*) distribuídas entre as tarefas (processos).
20. **Interface Humano-Computador (ICH):** parte das Ciências Exatas que se preocupa com a interação entre o homem e o computador, criando interfaces para tornar o computador em um ambiente simples e humanizado.

REFERÊNCIAS

DEITEL, P. J.; DEITEL, H. M.; CHOFFNES, D. R. **Sistemas Operacionais**. 3. ed. Pearson, 2017.

HILL, B. M.; BACON, J. **O Livro Oficial do Ubuntu**. 2. ed. Porto alegre: Bookman, 2008.

LINUX.ORG. Disponível em: <https://www.linux.org/>. Acesso em: 24 nov. 2023.

MACHADO, F. B.; MAIA, P. L. **Fundamentos de Sistemas Operacionais**. LTC, 2011. BVMB

NEMETH, E.; SNYDER, G.; HEIN T. R. **Manual Completo do Linux**: guia do administrador. 2. ed. São Paulo: Pearson Prentice Hall, 2007. (BVP)

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. BVMB.

TANENBAUM, A. S. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson Prentice Hall, 2008.